



Reconhecimento de padrões de falhas em etiquetas de códigos de barras: um sistema de visão computacional com orquestração multiagente

Recognition of failure patterns in barcode labels: a computer vision system with multi-agent orchestration

Saymon Coppi de Oliveira Silva*

2026

Resumo

A leitura confiável de códigos de barras depende da qualidade de impressão das etiquetas, avaliada por normas como a ISO/IEC 15416; contudo, os verificadores certificados são caros e reportam apenas uma nota, sem indicar a causa física do defeito nem a ação corretiva. Este trabalho propõe e desenvolve um sistema de baixo custo que, a partir de uma imagem capturada por um dispositivo comum, detecta defeitos típicos de impressão em etiquetas de códigos de barras e sugere sua causa provável. A solução combina visão computacional — uma rede neural convolucional com *backbone* leve, treinada por transferência de aprendizado — e orquestração multiagente com o *Agent Development Kit* (ADK), decompondo a tarefa em leitura/decodificação, estimativa de indicadores de qualidade, detecção de defeitos físicos e diagnóstico de causa-raiz fundamentado na documentação técnica do fabricante. O sistema é exposto por uma *API* única, consumida por um aplicativo Android e por uma interface web. A avaliação prevista compara a arquitetura multiagente a uma abordagem monolítica, quantificando o benefício da decomposição, e emprega um conjunto de dados híbrido, com imagens reais de captura e defeitos de impressão gerados sinteticamente de forma controlada

*Universidade Federal de São Paulo (Unifesp), Instituto de Ciência e Tecnologia. Programa de Pós-Graduação em Ciência da Computação. Disciplina: Visão Computacional (cód. 2587), Prof. Dr. Fabio Augusto Menocci Cappabianco. E-mail: saymoncoppi@gmail.com.

e reprodutível. Espera-se demonstrar a viabilidade de um apoio à decisão que vai além da nota, indicando causa provável e correção sugerida.

Palavras-chave: código de barras; qualidade de impressão; visão computacional; aprendizado profundo; sistemas multiagentes.

Abstract

Reliable barcode reading depends on the print quality of the labels, which is assessed by standards such as ISO/IEC 15416; however, certified verifiers are expensive and report only a grade, without indicating the physical cause of the defect or the corrective action. This work proposes and develops a low-cost system that, from an image captured by an ordinary device, detects typical printing defects in barcode labels and suggests their probable cause. The solution combines computer vision — a convolutional neural network with a lightweight backbone, trained by transfer learning — and multi-agent orchestration with the Agent Development Kit (ADK), decomposing the task into reading/decoding, quality-indicator estimation, physical-defect detection and root-cause diagnosis grounded in the manufacturer's technical documentation. The system is exposed through a single API, consumed by an Android application and by a web interface. The planned evaluation compares the multi-agent architecture with a monolithic baseline, quantifying the benefit of decomposition, and uses a hybrid dataset with real capture images and printing defects generated synthetically in a controlled and reproducible way. The expected outcome is to demonstrate the feasibility of a decision-support tool that goes beyond the grade, pointing to a probable cause and a suggested correction.

Keywords: barcode; print quality; computer vision; deep learning; multi-agent systems.

Sumário

1	Introdução	4
1.1	Contextualização	4
1.2	Problema de pesquisa	4
1.3	Objetivos	5
1.4	Contribuições do trabalho	5
1.5	Organização do artigo	6
2	Fundamentação teórica	6
2.1	Códigos de barras e simbologias	6
2.2	Qualidade de impressão e verificação (ISO/IEC 15416 e 15415)	6
2.3	Defeitos típicos de impressão térmica e suas causas	8
2.4	Processamento de imagens e visão computacional para inspeção	8
2.5	Sistemas multiagentes e o <i>Agent Development Kit</i> (ADK)	9
3	Trabalhos relacionados	10
3.1	Verificação normativa e diagnóstico de defeitos	10
3.2	Inspeção visual de defeitos com aprendizado profundo	10
3.3	Sistemas multiagentes aplicados a tarefas de visão	10
3.4	Lacuna que este trabalho preenche	10

4	Metodologia	11
4.1	Tipo de pesquisa	11
4.2	Conjunto de dados	11
4.3	Ferramentas utilizadas	13
4.4	Ambiente experimental	13
4.5	Métricas de avaliação	13
5	Proposta da solução	14
5.1	Arquitetura da solução	14
5.2	Fluxograma do processamento	14
5.3	Algoritmos empregados	15
5.4	Detalhes da implementação	15
5.5	Interface de uso: chat de inspeção	17
6	Resultados e discussão	18
6.1	Configuração dos experimentos	18
6.2	Resultados quantitativos	18
6.3	Multiagente <i>versus</i> abordagem monolítica	19
6.4	Análise e discussão	19
6.5	Limitações	20
7	Conclusão	20

Referências Bibliográficas	20
---	-----------

1 Introdução

1.1 Contextualização

Códigos de barras são a espinha dorsal da identificação automática de itens em varejo, logística, saúde e manufatura. Uma etiqueta é lida dezenas de vezes ao longo da cadeia — na expedição, no transporte, no recebimento e no ponto de venda — e cada leitura pressupõe que a impressão preserve as proporções de barras e espaços dentro de tolerâncias estreitas. Quando a impressão degrada, o custo não é apenas uma releitura: há reproprocessamento manual, atraso operacional, devolução de cargas e, em setores regulados, não conformidade.

Por essa criticidade, a qualidade de impressão é normatizada. A norma ISO/IEC 15416 define, para símbolos lineares (1D), um conjunto de parâmetros objetivos derivados do *perfil de refletância de varredura* — entre eles contraste do símbolo, modulação, decodificabilidade e defeitos — que são convertidos em notas de A a F (ISO/IEC, 2016). Para símbolos bidimensionais (2D), a ISO/IEC 15415 cumpre papel análogo (ISO/IEC, 2024), e a conformidade dos aparelhos verificadores é regida pela ISO/IEC 15426 (ISO/IEC, 2006). Na prática industrial, essa avaliação é feita por *verificadores ópticos* dedicados, equipamentos calibrados e de custo elevado.

Paralelamente, no chão de fábrica, os defeitos de impressão têm origem física bem conhecida e documentada pelos fabricantes. No caso da Zebra Technologies — referência deste trabalho —, os guias das impressoras industriais da série ZT (ZT411/ZT421) e a base de conhecimento associam sintomas visuais a causas mecânicas concretas: nível de *darkness* (temperatura) incorreto, pressão desigual da cabeça de impressão, enrugamento ou rompimento do *ribbon*, cabeça de impressão suja ou com elemento queimado, rolete (*platen*) desgastado ou sujo, e incompatibilidade entre mídia e *ribbon* (Zebra Technologies, 2024c). Ou seja, existe um corpo de conhecimento que liga aparência do defeito, componente responsável e ação corretiva.

1.2 Problema de pesquisa

Há uma lacuna entre esses dois mundos. Os verificadores certificados são caros, pouco portáteis e, sobretudo, reportam uma nota, não a causa-raiz: informam que o símbolo é grau “C” ou “D”, mas não dizem ao operador *por que* — se é *ribbon* enrugado, pressão desigual ou cabeça queimada — nem *o que fazer*. Do outro lado, o conhecimento de causa-raiz existe, mas está disperso em manuais e depende da experiência de um técnico para ser aplicado.

Este trabalho investiga a seguinte questão de pesquisa:

É possível, a partir de uma imagem capturada por um dispositivo comum, identificar defeitos típicos de impressão de códigos de barras e sugerir sua causa provável, combinando visão computacional e um sistema multiagente orquestrado, com precisão suficiente para apoiar a decisão de um operador?

Escopo do diagnóstico. O sistema **não é um verificador certificado** e não substitui a medição conforme ISO/IEC 15416. Ele atua como apoio à decisão: (i) classifica defeitos físicos de impressão e (ii) estima indicadores *inspirados* nos parâmetros da norma (contraste, uniformidade), sempre rotulados como aproximação, nunca como grau oficial.

1.3 Objetivos

Objetivo geral. Desenvolver e avaliar um sistema, exposto por uma API única e consumido por um aplicativo Android e por uma interface web, capaz de detectar defeitos comuns de impressão em etiquetas de códigos de barras e sugerir a causa provável, empregando visão computacional e orquestração multiagente com o *Agent Development Kit* (ADK).

Objetivos específicos.

- a) caracterizar os defeitos de impressão mais frequentes em impressoras térmicas industriais (referência: Zebra série ZT) e mapeá-los para suas causas físicas;
- b) construir/organizar um conjunto de imagens rotuladas contemplando as classes de defeito e a classe “sem defeito”;
- c) projetar uma arquitetura multiagente que decomponha a tarefa em leitura/decodificação, estimativa de indicadores de qualidade, detecção de defeitos físicos e diagnóstico de causa-raiz fundamentado na documentação do fabricante;
- d) implementar a solução e disponibilizá-la por uma API consumível por aplicativo e web; e
- e) avaliar o desempenho do sistema e verificar se a decomposição multiagente traz ganho mensurável frente a uma abordagem monolítica.

Escopo de símbolos. O trabalho contempla **1D** (Code 128, Code 39, EAN-13/EAN-8, *Interleaved 2 of 5*, Pharmacode) e **2D** (Data Matrix, QR Code). A ênfase de qualidade recai sobre a ISO/IEC 15416 (1D) e a ISO/IEC 15415 (2D). O conjunto de imagens reais atualmente disponível cobre apenas 1D; as amostras 2D e as amostras de defeito de impressão são obtidas por geração sintética controlada (Seção 4.2).

1.4 Contribuições do trabalho

As principais contribuições são:

- a) um **mapeamento sistematizado** de defeitos visuais de impressão para causas físicas e ações corretivas, consolidado a partir da documentação técnica da Zebra (guia da ZT411/ZT421, manual de manutenção e base de conhecimento);
- b) uma **arquitetura multiagente** (ADK) que separa leitura, estimativa de indicadores, detecção de defeitos e diagnóstico de causa-raiz, servida por uma API única para aplicativo Android e web;
- c) um **agente de diagnóstico fundamentado** (com recuperação sobre a documentação do fabricante) que vai além da nota e sugere causa provável e correção;
- d) uma **avaliação comparativa** entre a arquitetura multiagente e uma abordagem monolítica, quantificando o benefício da decomposição; e
- e) um **conjunto de imagens híbrido**: imagens reais de códigos de barras capturados em condições variadas (52 amostras 1D organizadas por simbologia) combinadas a um conjunto de defeitos de impressão gerados sinteticamente de forma controlada e reproduzível.

1.5 Organização do artigo

A Seção 2 apresenta a fundamentação teórica. A Seção 3 revisa trabalhos relacionados e delimita a lacuna. A Seção 4 descreve a metodologia. A Seção 5 detalha a solução proposta. A Seção 6 apresenta e discute os resultados. A Seção 7 conclui e aponta trabalhos futuros.

2 Fundamentação teórica

2.1 Códigos de barras e simbologias

Um código de barras codifica dados na largura e no arranjo de elementos claros e escuros. Nos símbolos **lineares (1D)**, como Code 128, EAN-13 e GS1-128, a informação está na sequência horizontal de barras e espaços de larguras variáveis; a menor largura nominal é chamada **dimensão X** (ou módulo) e serve de referência para todas as tolerâncias. O símbolo é delimitado por **zonas de silêncio** (áreas claras) obrigatórias antes e depois das barras, sem as quais o leitor não isola o código. Nos símbolos **bidimensionais (2D)**, como Data Matrix e QR Code, os dados se distribuem em uma matriz de células, permitindo maior densidade e correção de erros.

A leitura pressupõe que as proporções impressas correspondam às nominais. Pequenos desvios sistemáticos — barras mais largas (ganho) ou mais estreitas (perda) que o previsto — comprometem a decodificação. É exatamente esse tipo de desvio que a impressão térmica pode introduzir.

2.2 Qualidade de impressão e verificação (ISO/IEC 15416 e 15415)

A avaliação normativa parte do **perfil de refletância de varredura** (*scan reflectance profile*, SRP): uma linha de varredura atravessa o símbolo medindo a refletância ao longo do percurso, produzindo uma curva de picos (espaços claros) e vales (barras escuras). Todos os parâmetros de nota derivam desse perfil (ISO/IEC, 2016).

Para símbolos lineares, a ISO/IEC 15416 avalia nove atributos por varredura, entre os quais se destacam os apresentados na Tabela 1.

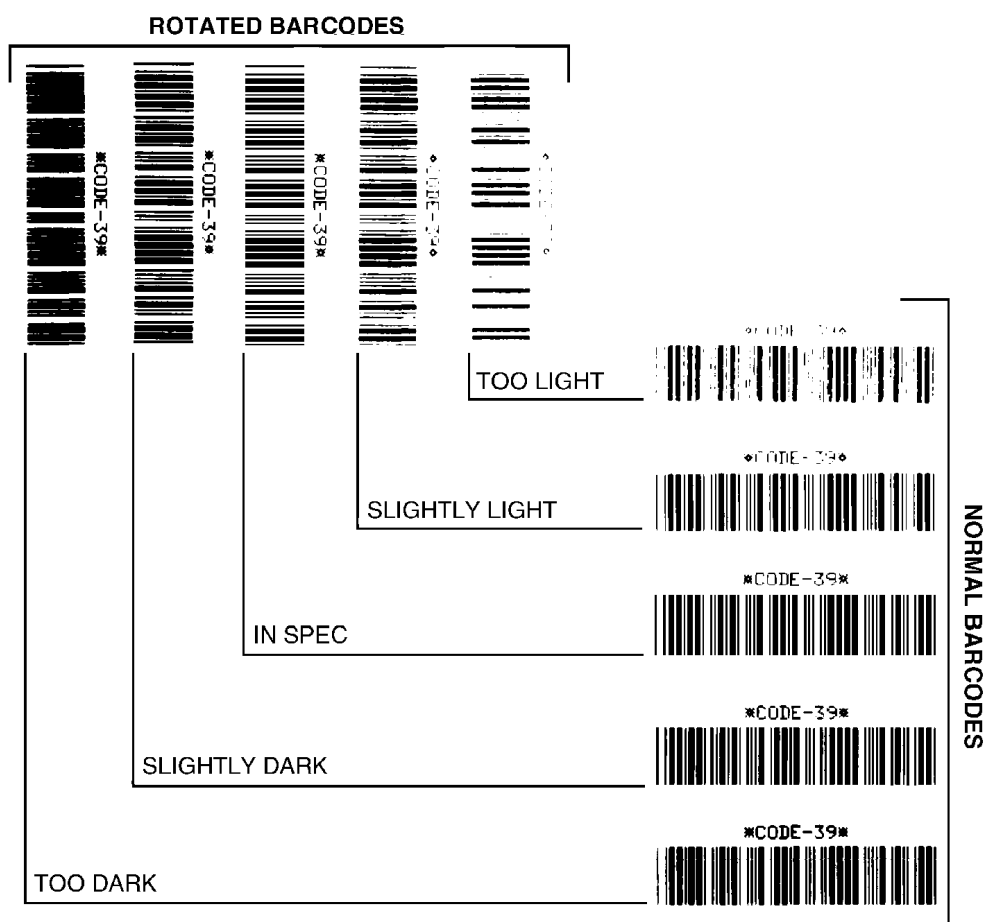
Tabela 1 – Parâmetros de qualidade da ISO/IEC 15416 e sua relação com o defeito de impressão

Parâmetro	O que mede	Relação com o defeito de impressão
Contraste do símbolo (SC)	Diferença entre a maior e a menor refletância	Impressão clara ou <i>mídia-ribbon</i> incompatível reduzem o contraste
Contraste de borda (EC)	Contraste entre barra e espaço adjacentes	Bordas “borradas” por pressão/velocidade inadequadas
Modulação (MOD)	Uniformidade dos elementos ao longo do símbolo	Pressão desigual e sujeira geram variação
Defeitos (ERN)	Maior não uniformidade de refletância de um elemento	Manchas, <i>voids</i> e pontos queimados
Decodificabilidade	Margem do símbolo frente ao algoritmo de referência	Ganho/perda de largura das barras

Fonte: elaboração própria (2026), a partir de ISO/IEC (2016).

O ajuste de *darkness* (temperatura da cabeça) é um dos fatores que mais afetam esses parâmetros. A Figura 1 ilustra a transição da impressão clara demais (*too light*), em que as barras finas desaparecem, à escura demais (*too dark*), em que barras e espaços se fundem, passando pela faixa dentro da especificação (*in spec*).

Figura 1 – Efeito do *darkness* na qualidade do código de barras: do claro demais (*too light*) ao escuro demais (*too dark*)



Fonte: [Zebra Technologies \(2024b\)](#).

A nota de uma varredura é a **menor** entre os parâmetros; a nota do símbolo é a **média de dez varreduras** distribuídas ao longo da altura. Para 2D, a ISO/IEC 15415 cumpre papel equivalente com parâmetros adaptados à matriz (ISO/IEC, 2024), e a marcação direta de peça (DPM) é tratada pela ISO/IEC 29158 (ISO/IEC, 2020).

Ponto metodológico importante. A verificação *conformante* exige condições ópticas controladas — fonte de luz definida, abertura de medição, geometria e calibração de refletância — providas por verificadores certificados (ISO/IEC, 2006). Uma imagem de câmera comum não reproduz essas condições. Por isso, neste trabalho os parâmetros da norma são usados como referencial conceitual e como indicadores aproximados, não como grau oficial — o que delimita honestamente o escopo (Seção 2).

2.3 Defeitos típicos de impressão térmica e suas causas

Impressoras térmicas de transferência (como a Zebra ZT411/ZT421) formam a imagem aquecendo seletivamente uma cabeça de impressão que transfere tinta do *ribbon* para a mídia, prensada pelo rolete (*platen*). Cada componente falha de um modo com assinatura visual característica. A partir do guia do usuário da ZT411/ZT421, do procedimento de manutenção e da base de conhecimento da Zebra ([Zebra Technologies, 2024c](#)), consolida-se o mapeamento da Tabela 2 — que é, em si, uma das contribuições do trabalho.

Tabela 2 – Mapeamento entre aparência do defeito, causa provável e ação corretiva

Defeito (aparência)	Causa provável	Ação corretiva típica
Código não escaneia; imagem “suja”	<i>Darkness</i> alto demais ou pressão da cabeça incorreta	Reduzir o <i>darkness</i> ; reduzir velocidade; ajustar pressão
Impressão clara (<i>faded</i>) em toda a etiqueta	<i>Darkness</i> baixo, mídia/ <i>ribbon</i> incompatível ou alta velocidade	Elevar <i>darkness</i> ; trocar para combinação mídia- <i>ribbon</i> adequada
Impressão clara/escuro em um lado da etiqueta	Pressão desigual da cabeça	Ajustar a pressão da cabeça (<i>toggles</i>)
Linhas cinza finas e angulares em etiquetas em branco	<i>Ribbon</i> enrugado	Corrigir tensão/alinhamento do <i>ribbon</i>
Trilhas longas de impressão faltando	Elemento da cabeça danificado (linhas brancas verticais)	Acionar assistência técnica (troca de cabeça)
<i>Voids</i> no código; qualidade inconsistente	Cabeça de impressão suja	Limpar cabeça e rolete com álcool isopropílico 99,7%
Manchas (<i>smudge</i>)	Mídia/ <i>ribbon</i> inadequados para alta velocidade	Trocar por insumos recomendados
Perda de registro / deslocamento vertical	Roleta suja, mídia mal carregada, sensor descalibrado	Limpar rolete; recarregar mídia; calibrar sensores

Fonte: elaboração própria (2026), a partir de [Zebra Technologies \(2024c\)](#).

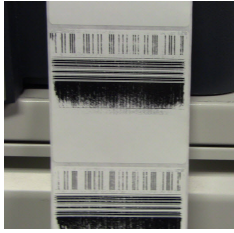
A Figura 2 exemplifica, em etiquetas impressas reais, as assinaturas visuais de vários desses defeitos, tal como documentadas pela Zebra ([Zebra Technologies, 2024a](#)). Note-se a correspondência com as classes reproduzidas sinteticamente na Seção 4.2.

A manutenção preventiva atua sobre as causas de origem: a limpeza periódica da cabeça e do rolete com álcool isopropílico 99,7% (ou kit de manutenção Zebra) é indicada quando surgem *voids*, e a documentação alerta que *darkness* excessivo desgasta a cabeça prematuramente e pode “queimar” o *ribbon* ([Zebra Technologies, 2024c](#)). Esse conhecimento é o que alimenta o agente de diagnóstico proposto na Seção 5.

2.4 Processamento de imagens e visão computacional para inspeção

O reconhecimento dos defeitos acima é um problema de **inspeção visual automatizada**. As etapas clássicas envolvem:

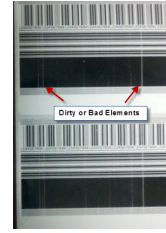
Figura 2 – Assinaturas visuais reais de defeitos de impressão térmica em etiquetas de códigos de barras



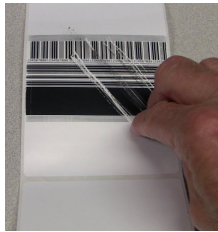
(a) Impressão clara (*faded*)



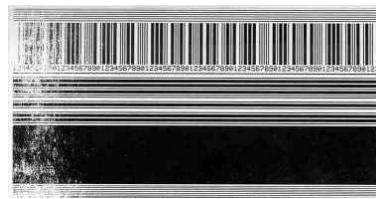
(b) *Ribbon* enrugado



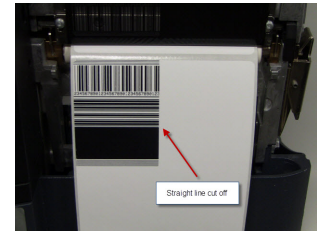
(c) Elemento da cabeça danificado



(d) Mancha (*smear*)



(e) Pressão desigual da cabeça



(f) Perda de registro (*cutoff*)

Fonte: [Zebra Technologies \(2024a\)](#).

- a) **pré-processamento** — conversão de cor, correção de iluminação e realce de contraste para atenuar variações de captura ([GONZALEZ; WOODS, 2018](#));
- b) **localização/segmentação do símbolo** — isolar a região do código na etiqueta, condição para medir refletância e detectar defeitos localizados ([SZELISKI, 2022](#));
- c) **deteção de objetos/defeitos** — localizar e classificar regiões defeituosas; abordagens supervisionadas de um estágio (por exemplo, da família YOLO) são amplamente usadas em inspeção industrial;
- d) **classificação por redes convolucionais (CNN)** — atribuir a classe de defeito a partir de características aprendidas, dispensando extração manual de atributos; e
- e) **deteção de anomalias** — quando amostras de defeito real são escassas, métodos não supervisionados aprendem o “normal” e sinalizam desvios, contornando o desbalanceamento de dados.

A literatura recente de inspeção industrial converge nesses pilares e destaca um desafio recorrente diretamente relevante a este trabalho: a **escassez de imagens de defeito real**, que motiva aumento de dados e abordagens semi/não supervisionadas ([SHUKLA et al., 2025](#); [TAO et al., 2022](#)).

2.5 Sistemas multiagentes e o *Agent Development Kit* (ADK)

Um **sistema multiagente** é um conjunto de agentes autônomos que colaboram para uma meta comum, cada um responsável por uma subtarefa. O *Agent Development Kit* (ADK), apresentado pelo Google no Cloud NEXT 2025, é um *framework* de código aberto para construir e orquestrar esses sistemas ([Google, 2025b](#)). Ele organiza os agentes em três tipos:

- a) **agentes LLM** — o “raciocínio”, que interpreta entradas e decide ações;
- b) **agentes de *workflow*** — orquestradores de fluxo (sequencial, paralelo, em laço); e

- c) **agentes customizados** — especialistas que encapsulam lógica ou modelos próprios (por exemplo, um modelo de visão).

Os agentes se organizam em uma **hierarquia** e se comunicam por estado compartilhado, delegação e invocação explícita (Google, 2025a). A justificativa central do paradigma — e o argumento que este trabalho pretende testar empiricamente — é que decompor um problema em agentes especializados tende a ser mais confiável, modular e manutenível do que um único modelo/prompt monolítico, porque cada agente resolve uma tarefa mais simples e bem delimitada.

3 Trabalhos relacionados

3.1 Verificação normativa e diagnóstico de defeitos

A base da verificação de qualidade são as normas ISO/IEC 15416 e 15415, implementadas por verificadores comerciais que reportam notas A–F (ISO/IEC, 2016; ISO/IEC, 2024). Há também soluções que inferem falhas do mecanismo de impressão a partir da própria leitura — por exemplo, detectando linhas não impressas para gerar um relatório de manutenção do cabeçote e alertar antes da falha total (ACKLEY, 2017). **Limitação comum:** dependem de *hardware* dedicado e/ou entregam a nota/alerta, sem um diagnóstico de causa-raiz acionável e contextualizado pela documentação do fabricante.

3.2 Inspeção visual de defeitos com aprendizado profundo

A inspeção industrial baseada em aprendizado profundo é sistematizada em levantamentos recentes que cobrem CNNs, detecção de objetos e detecção de anomalias, com ênfase no problema de dados escassos (SHUKLA et al., 2025; TAO et al., 2022). Aplicações diretas ao domínio de impressão incluem a detecção de defeitos em tecidos estampados com CNN (JUN et al., 2021) e a detecção/localização de anomalias em manufatura aditiva por transferência de aprendizado (AVRO et al., 2024). **Limitação comum:** focam a classificação/localização do defeito visual, sem ligá-lo à causa física do processo nem a uma ação corretiva.

3.3 Sistemas multiagentes aplicados a tarefas de visão

A orquestração multiagente com ADK e *frameworks* correlatos vem sendo aplicada a tarefas complexas que se beneficiam de decomposição, tratando agentes especializados como ferramentas de um agente raiz (Google, 2025b). **Lacuna observada:** a combinação de orquestração multiagente com inspeção de qualidade de impressão de códigos de barras — em especial acoplando detecção visual a diagnóstico fundamentado em documentação técnica — é pouco explorada.

3.4 Lacuna que este trabalho preenche

Reunindo os três eixos: os verificadores dão nota mas não causa; os métodos de visão detectam o defeito mas não a causa nem a correção; e a orquestração multiagente é madura porém raramente aplicada a este domínio. Este trabalho se posiciona nessa interseção, propondo um sistema de baixo custo, servido por API, que combina estimativa de indicadores de qualidade, detecção visual de defeitos físicos e diagnóstico de causa-raiz fundamentado na documentação da Zebra,

orquestrados por agentes especializados. O Quadro comparativo da Tabela 3 posiciona este trabalho frente às abordagens existentes.

Tabela 3 – Comparativo entre abordagens quanto aos recursos oferecidos

Abordagem	Nota ISO	Defeito físico	Causa-raiz	Sem <i>hardware</i> dedicado
Verificadores certificados	Sim	Não	Não	Não
Visão computacional (CNN/anomalia)	Não	Sim	Não	Sim
Deteção via leitura (parte)	Parcial	Parcial	Parcial	Não
Este trabalho	Aprox.	Sim	Sim	Sim

Fonte: elaboração própria (2026).

4 Metodologia

4.1 Tipo de pesquisa

Pesquisa de natureza **aplicada**, abordagem **quantitativa** e objetivo **exploratório-experimental**: constrói-se um artefato (o sistema) e mede-se seu desempenho em condições controladas, com procedimento experimental.

4.2 Conjunto de dados

O problema tem duas naturezas distintas — a **legibilidade da captura** (a imagem do código está boa o suficiente para ler?) e o **defeito de impressão** (que falha do processo produziu a marca?) — e cada uma exige um tipo de dado. Por isso, adota-se uma estratégia de **duas trilhas**.

Trilha A — imagens reais de captura (base disponível). Conjunto de 52 imagens reais de códigos de barras fotografados em produtos e embalagens, em tons de cinza, sob ângulo, reflexo, curvatura e fundos variados, já organizadas por simbologia, conforme a Tabela 4. Esse conjunto é usado para treinar/avaliar o agente de leitura/decodificação e o agente de indicadores de legibilidade. Como são todas 1D, as amostras 2D são complementadas na Trilha B.

Trilha B — defeitos de impressão sintéticos (gerados). Para treinar o agente de defeitos físicos não há um conjunto público rotulado de defeitos de impressão térmica, e as fotos da base de conhecimento da Zebra são material protegido por direitos autorais. Adota-se, então, **geração sintética controlada**: a partir de códigos limpos (1D e 2D), aplicam-se transformações que reproduzem a assinatura visual de cada defeito descrito na documentação Zebra (Zebra Technologies, 2024c), conforme a Tabela 5. Um gerador de referência foi implementado (`gerar_dataset.py`) e produz as sete classes de forma balanceada, com divisão automática em treino/validação/teste (70/15/15) e parâmetros de defeito amostrados aleatoriamente em faixas definidas. A Figura 3 ilustra amostras das classes geradas.

Tabela 4 – Trilha A — imagens reais por simbologia

Simbologia	N.º de imagens
EAN-13	19
Code 39	6
Code 128	5
EAN-8	5
<i>Interleaved 2 of 5</i>	5
EAN-13 (<i>add-on</i> 2)	4
EAN-13 (<i>add-on</i> 5)	4
Pharmacode	4
Total	52

Fonte: elaboração própria (2026).

Tabela 5 – Trilha B — classes de defeito e transformação sintética correspondente

Classe de defeito	Transformação sintética	Sintoma reproduzido
Cabeça queimada (elemento danificado)	Linhas brancas verticais contínuas	Trilhas longas de impressão faltando
<i>Ribbon</i> enrugado	Faixas claras finas e diagonais	Linhas cinza angulares
Ponto queimado / <i>darkness</i> alto	Manchas escuras localizadas	Excesso de temperatura
Impressão clara (<i>darkness</i> baixo)	Redução de contraste global	<i>Faded</i> / mídia- <i>ribbon</i> incompatível
Pressão desigual	Borrão direcional (gradiente em um lado)	Densidade não uniforme
Cabeça suja (<i>voids</i>)	Falhas brancas pontuais nas barras	<i>Void</i> s no código
Sem defeito	(código limpo)	Classe de controle

Fonte: elaboração própria (2026).

Essa abordagem é reprodutível, balanceável e livre de restrições autorais, e é prática consolidada na literatura de inspeção quando amostras reais são escassas (SHUKLA et al., 2025; TAO et al., 2022).

Divisão e aumento de dados. Cada trilha é dividida em treino/validação/teste (70/15/15), estratificando por classe. Sobre a Trilha A aplica-se aumento de dados clássico (rotação, variação de brilho/contraste, recortes) para ampliar a variabilidade de captura. Os parâmetros dos defeitos sintéticos (intensidade, quantidade, posição) são amostrados aleatoriamente em faixas definidas, para evitar que o modelo aprenda um padrão fixo.

Datasets externos (complementares). Para avaliar a generalização para imagens reais e ampliar a variabilidade, o conjunto pode ser complementado por bases públicas de defeitos de códigos de barras e etiquetas disponíveis no *Roboflow Universe*¹ — empregadas preferencialmente como conjunto de teste de generalização (sintético → real). O projeto inclui um utilitário de *download* (`baixar_roboflow.py`) para essa finalidade.

¹ Busca por conjuntos de dados de defeitos: <<https://universe.roboflow.com/search?q=barcode+damage>>; termos correlatos: *barcode defect* e *label print quality*.

4.3 Ferramentas utilizadas

- a) **visão computacional:** Python, OpenCV e **PyTorch**. O PyTorch é hoje o *framework* predominante na comunidade acadêmica, o que favorece a reprodutibilidade e a comparação com trabalhos correlatos. Para atender a máquinas modestas, adota-se transferência de aprendizado com um *backbone* leve (**MobileNetV3-Small** ou **EfficientNet-B0**), que treina bem em GPU de entrada ou mesmo CPU; como a inferência é servida pela API (lado servidor), o dispositivo do usuário não precisa executar o modelo;
- b) **decodificação e OCR:** pyzbar (ZBar) para decodificar o símbolo e verificar a legibilidade, e o **Tesseract** (pytesseract) para o reconhecimento óptico do texto humano-legível impresso junto às barras;
- c) **orquestração multiagente:** *Agent Development Kit* (ADK) (Google, 2025b), com **Gemini** como modelo LLM dos agentes de raciocínio, aproveitando a integração nativa do ADK com o ecossistema Google;
- d) **entrega:** API única (FastAPI) que serve um **chat** web internacionalizado (português/inglês) para o envio da imagem e que pode, também, alimentar um aplicativo Android; e
- e) **escrita e gestão bibliográfica:** L^AT_EX (abnTeX2/Overleaf) e Zotero.

As técnicas empregadas correspondem diretamente ao conteúdo da disciplina de Visão Computacional em que este trabalho se insere — aquisição e representação de imagens com OpenCV; limiarização e filtragem; detecção de arestas e contornos; segmentação; morfologia matemática; descritores e características; classificadores; e redes neurais convolucionais. O projeto, assim, exercita de ponta a ponta o pipeline clássico de visão computacional, culminando em uma CNN para a classificação dos defeitos.

4.4 Ambiente experimental

Os experimentos foram executados em **CPU** (Intel Core i7-1185G7, 8 *threads*, 31 GB de RAM), **sem GPU dedicada**, com Python 3.12, PyTorch 2.13 (CPU), *torchvision* 0.28 e OpenCV 4.10; o ambiente Python é gerenciado com *uv*. O *fine-tuning* da CNN sobre o conjunto sintético levou poucos minutos nessa configuração, evidenciando a viabilidade em máquinas modestas. *Observação:* como o contraste medido depende fortemente da captura, padronizar iluminação e distância é parte do rigor experimental para as imagens reais.

4.5 Métricas de avaliação

- a) **classificação de defeitos:** acurácia, precisão, revocação e F1 por classe, além da matriz de confusão;
- b) **detecção/localização (se houver):** mAP e IoU;
- c) **indicadores de qualidade:** concordância entre o indicador estimado e a referência disponível (verificador, se houver, ou rótulo especialista); e
- d) **benefício do multiagente:** comparação das métricas acima entre a arquitetura multi-agente e a abordagem monolítica (Seção 6.3), além de latência e interpretabilidade do laudo.

5 Proposta da solução

5.1 Arquitetura da solução

A solução adota o padrão **hierárquico** do ADK (Google, 2025a), no qual um agente raiz coordena agentes especializados. A tarefa “avaliar uma etiqueta” é decomposta em quatro análises especializadas e uma etapa de síntese, explorando dois recursos de orquestração do ADK: execução paralela para as análises independentes e execução sequencial para as etapas que dependem das anteriores. O sistema é composto por:

- a) **agente de leitura/decodificação** — com OpenCV, segmenta a região do código (gradiente, limiarização e morfologia); decodifica com o **pyzbar** (simbologia e conteúdo) e lê o texto humano-legível com o Tesseract (OCR); retorna legível/ilegível e o conteúdo;
- b) **agente de indicadores de qualidade** — sobre a região segmentada pelo OpenCV, estima indicadores inspirados na ISO/IEC 15416 (contraste, uniformidade e nitidez), sempre como aproximação (Seção 2.2);
- c) **agente de defeitos físicos** — classifica o defeito de impressão (as sete classes da Seção 4.2) usando a CNN treinada, exposta ao ADK como ferramenta;
- d) **agente de diagnóstico** — agente LLM (Gemini) que, com os resultados anteriores e uma ferramenta de recuperação sobre a documentação Zebra (RAG), infere a causa provável e a ação corretiva; e
- e) **agente de laudo** — consolida tudo em um JSON estruturado devolvido pela API.

Os agentes 1–3 são independentes e rodam em paralelo (**ParallelAgent**); o agente 4 depende das saídas dos três e o agente 5 depende do 4, formando uma cadeia (**SequentialAgent**). A comunicação entre agentes usa o estado compartilhado da sessão: cada agente grava seu resultado em uma chave (**output_key**) e os seguintes leem essas chaves.

5.2 Fluxograma do processamento

O fluxo ponta a ponta é: envio da imagem no chat → POST **/analisar** na API → pré-processamento e segmentação com OpenCV → **verificação da presença** de um código de barras → (havendo código) **ParallelAgent** executa, em paralelo, a leitura/decodificação (**pyzbar** + OCR Tesseract), os indicadores de qualidade e a classificação do defeito (CNN) → o agente de diagnóstico consome as três saídas e consulta a documentação Zebra → o agente de laudo agrega em JSON → resposta única → renderização no chat.

Decisão de projeto: curto-circuito de presença do código. A verificação da presença do código de barras ocorre **antes** da inspeção pesada. Se nenhum código é detectado na imagem, o sistema retorna um laudo **mínimo** — apenas o aviso “Nenhum código de barras detectado na imagem.” —, sem executar a CNN de classificação nem invocar o agente LLM. A motivação é dupla: (i) **evitar diagnósticos espúrios**, isto é, não apresentar ao operador uma causa/correção plausível para uma imagem que sequer contém um código (falsa sensação de resultado útil); e (ii) **economizar processamento e chamadas de API** ao Gemini, que só são disparados quando há, de fato, o que inspecionar. A legibilidade bem-sucedida já implica presença; caso contrário, um detector leve sobre a imagem em tons de cinza decide o curto-circuito.

5.3 Algoritmos empregados

- a) **segmentação do símbolo:** localização da região do código e das zonas de silêncio (limiarização adaptativa mais operações morfológicas, ou um detector leve treinado);
- b) **estimativa de indicadores:** a partir da imagem em tons de cinza, extração de perfis de refletância aproximados e cálculo de contraste e de medidas de uniformidade por elemento — apresentados como indicadores, não como grau ISO oficial;
- c) **classificação de defeitos:** CNN com *backbone* leve (MobileNetV3-Small ou EfficientNet-B0) e transferência de aprendizado, treinada no conjunto sintético da Seção 4.2 e testada na generalização para imagens reais; e
- d) **diagnóstico:** recuperação (RAG) sobre a base de conhecimento estruturada (Seção 2.3) e raciocínio do agente LLM para ordenar as causas prováveis.

5.4 Detalhes da implementação

Cada especialista é definido como um agente do ADK; os modelos e rotinas de visão computacional (OpenCV, pyzbar, Tesseract e a CNN em PyTorch) são expostos como **ferramentas** (funções Python) que o ADK encapsula automaticamente. O código é organizado no pacote *inspetor* (módulos *visao*, *rede*, *kb*, *diagnostico*, *ferramentas* e *agentes*) e na camada *app* (CLI, API e chat web). O trecho da Listagem 1 é uma implementação de referência (a API do ADK evolui — consultar a documentação oficial ([Google, 2025a](#))).

```
1 # agentes.py -- implementacao de referencia (Google ADK + Gemini)
2 from google.adk.agents import LlmAgent, ParallelAgent, SequentialAgent
3
4 MODEL = "gemini-flash-latest"
5
6 # ---- Ferramentas: rotinas de visao computacional (loais) ----
7 def decodificar_codigo(caminho_imagem: str) -> dict:
8     """Localiza e decodifica o codigo. Retorna simbologia, conteudo e se e legivel."""
9     from inspetor.visao import decodificar # OpenCV + pyzbar
10    return decodificar(caminho_imagem) # {"legivel": bool, "simbologia": str, "conteudo": str}
11
12 def estimar_indicadores(caminho_imagem: str) -> dict:
13     """Estima indicadores aproximados inspirados na ISO/IEC 15416 (nao e grau oficial)."""
14     from inspetor.visao import indicadores
15     return indicadores(caminho_imagem) # {"contraste": float, "uniformidade": float}
16
17 def classificar_defeito(caminho_imagem: str) -> dict:
18     """Classifica o defeito de impressao com a CNN treinada (7 classes)."""
19     from inspetor.rede import prever # checkpoint MobileNetV3-Small (PyTorch)
20     return prever(caminho_imagem) # {"classe": str, "confianca": float}
21
22 def buscar_documentacao_zebra(sintomas: str) -> str:
23     """Recupera trechos relevantes da base de conhecimento estruturada (RAG)."""
24     from inspetor.kb import buscar
25     return buscar(sintomas)
26
27 # ---- Agentes especialistas (analises independentes -> paralelo) ----
```

```

28 agente_leitura = LlmAgent(
29     name="leitura", model=MODEL, tools=[decodificar_codigo],
30     instruction="Chame decodificar_codigo com a imagem e reporte se o codigo e legivel.",
31     output_key="leitura",
32 )
33 agente_indicadores = LlmAgent(
34     name="indicadores", model=MODEL, tools=[estimar_indicadores],
35     instruction="Chame estimar_indicadores e reporte contraste e uniformidade.",
36     output_key="indicadores",
37 )
38 agente defeito = LlmAgent(
39     name="defeito", model=MODEL, tools=[classificar_defeito],
40     instruction="Chame classificar_defeito e reporte a classe e a confianca.",
41     output_key="defeito",
42 )
43 analise_paralela = ParallelAgent(
44     name="analise_paralela",
45     sub_agents=[agente_leitura, agente_indicadores, agente_defeito],
46 )
47
48 # ---- Diagnostico (depende das analises) ----
49 agente_diagnostico = LlmAgent(
50     name="diagnostico", model=MODEL, tools=[buscar_documentacao_zebra],
51     instruction=(
52         "Dado o defeito detectado ({defeito}), a legibilidade ({leitura}) e os "
53         "indicadores ({indicadores}), use buscar_documentacao_zebra para fundamentar "
54         "e proponha a CAUSA PROVAVEL e a ACAO CORRETIVA, citando o trecho de origem."
55     ),
56     output_key="diagnostico",
57 )
58
59 # ---- Laudo final (JSON estruturado) ----
60 agente_laudo = LlmAgent(
61     name="laudo", model=MODEL,
62     instruction=(
63         "Consolide {leitura}, {indicadores}, {defeito} e {diagnostico} em um JSON com as "
64         "chaves: legivel, simbologia, indicadores, defeito, causa_provavel, "
65         "correcao_sugerida, confianca_geral. Responda SOMENTE o JSON."
66     ),
67     output_key="laudo",
68 )
69
70 # ---- Orquestrador raiz: paralelo -> diagnostico -> laudo ----
71 root_agent = SequentialAgent(
72     name="inspetor_etiqueta",
73     sub_agents=[analise_paralela, agente_diagnostico, agente_laudo],
74 )

```

Listing 1 – agentes.py — implementação de referência (Google ADK + Gemini)

A API expõe um **endpoint único** que devolve o laudo e serve a página de chat: a mesma porta atende o chat web e pode atender um aplicativo Android (Listagem 2). Além da imagem, o endpoint recebe do chat dois campos de formulário — o idioma do laudo (pt-BR/en-US) e o

inspetor (motor de diagnóstico) —, que são repassados ao pipeline; a página, por sua vez, recebe do servidor o nome do modelo LLM ativo, injetado na resposta. Por robustez, a Listagem 2 usa o **pipeline direto** (`analisar_imagem`), que funciona mesmo sem chave do Gemini; o grafo do ADK (Listagem 1) é a variante orquestrada.

```
1 # api.py -- FastAPI: serve o chat e o endpoint unico de analise
2 from fastapi import FastAPI, UploadFile, File, Form
3 from fastapi.responses import HTMLResponse, JSONResponse
4 from inspetor.ferramentas import analisar_imagem # pipeline direto (robusto)
5
6 app = FastAPI()
7
8 @app.get("/") # pagina de chat (envio da imagem)
9 def chat():
10     pagina = open("app/chat.html", encoding="utf-8").read()
11     return HTMLResponse(injetar_modelo_ativo(pagina)) # informa o LLM ativo a UI
12
13 @app.post("/analisar-hmtx") # idioma + motor escolhidos na UI
14 async def analisar(imagem: UploadFile = File(...),
15                    idioma: str = Form("pt-BR"), # pt-BR / en-US (laudo bilingue)
16                    inspetor: str = Form("llm")): # llm / kb / auto
17     caminho = await salvar_temp(imagem) # persiste a imagem enviada
18     usar_gemini = inspetor != "kb" # "kb" forca as regras Zebra
19     laudo = analisar_imagem(caminho, usar_gemini=usar_gemini, idioma=idioma)
20     return HTMLResponse(cartao_laudo_html(laudo, idioma)) # cartao do laudo (chat)
```

Listing 2 – api.py — FastAPI (esboço)

A Listagem 3 mostra o esquema do laudo (resposta da API).

```
1 {
2     "legivel": true,
3     "codigo_detectado": true,
4     "simbologia": "CODE128",
5     "conteudo": "CB123",
6     "texto_ocr": "CB123",
7     "indicadores": { "contraste": 0.82, "uniformidade": 0.74, "nitidez": 0.60 },
8     "defeito": { "classe": "ribbon_enrugado", "classe_pt": "Ribbon enrugado",
9                 "confianca": 0.91 },
10    "causa_provavel": "Carregamento/tensao incorretos do ribbon (enrugamento)",
11    "correcao_sugerida": "Verificar tensao, alinhamento e percurso do ribbon",
12    "fonte": "Zebra Technologies (2024)",
13    "via_diagnostico": "regras",
14    "confianca_geral": 0.91
15 }
```

Listing 3 – Esquema do laudo devolvido pela API (JSON)

5.5 Interface de uso: chat de inspeção

A solução é entregue como um **chat**: o usuário envia a foto de uma etiqueta e recebe, em resposta, um cartão de laudo estruturado. A mesma API atende uma página web de chat e

uma interface de linha de comando, e o grafo do ADK também pode ser exposto pelo utilitário **adk web**. Por robustez, o sistema oferece dois caminhos com a mesma saída: o **pipeline direto** (chamadas de função encadeadas, que funciona mesmo sem chave do Gemini, recorrendo às regras da base de conhecimento Zebra) e o **pipeline orquestrado** pelo ADK. O primeiro serve, ainda, de *baseline* monolítico na comparação da Seção 6.3. A Figura 4 mostra o sistema em funcionamento: a etiqueta enviada pelo usuário e o laudo devolvido, com a leitura do código decodificado, os indicadores de qualidade, o defeito classificado e a causa provável com a correção sugerida.

Configurações e internacionalização. A interface reúne suas preferências em um **menu de configurações** (ícone de três pontos), ilustrado na Figura 5, com quatro seções: (i) **Idioma** — português brasileiro ou inglês, com internacionalização de ponta a ponta, pois tanto os rótulos da interface quanto o texto do laudo (nomes de defeito, causa e correção) são traduzidos, respectivamente, no cliente e no servidor; (ii) **Tema** — claro, escuro ou automático (segue o sistema operacional); (iii) **Inspetor**, que seleciona o **motor de diagnóstico**: *LLM* (Gemini, exibindo o modelo ativo informado pelo servidor), *KB* (regras sobre a base de conhecimento Zebra) ou *Auto* (prefere o LLM e recorre à KB como *fallback*, opção oferecida apenas quando há um modelo configurado); e (iv) **Ações** — exportar a conversa em PDF ou limpar o histórico. As preferências persistem localmente no navegador, e o idioma e o motor escolhidos acompanham cada análise enviada à API. Essa escolha de motor dá ao usuário controle explícito sobre o compromisso entre o diagnóstico fundamentado pelo LLM e a resposta determinística por regras, útil inclusive quando não há conectividade ou chave de API disponível.

6 Resultados e discussão

6.1 Configuração dos experimentos

O experimento **E1 (classificação de defeitos)** foi executado sobre o conjunto sintético (Seção 4.2), dividido em 434 imagens de treino, 91 de validação e 105 de teste (as sete classes com 15 amostras cada no teste). Empregou-se a CNN **MobileNetV3-Small** inicializada com pesos do ImageNet e submetida a *fine-tuning* completo (*backbone* descongelado) por 40 épocas, com otimizador Adam (taxa de aprendizado 5×10^{-4}), perda de entropia cruzada e lotes de 32 imagens; seleciona-se o melhor modelo pela acurácia de validação. Os experimentos **E2** (concordância dos indicadores com uma referência) e **E3** (multiagente *versus* monolítico) dependem, respectivamente, de um verificador/rótulo de referência e de uma chave do modelo LLM, permanecendo em andamento (Seção 6.3).

6.2 Resultados quantitativos

Em uma execução real do sistema, a classificação de defeitos alcançou **acurácia de 81,9%** no conjunto de teste (105 imagens), com médias macro de precisão 0,85, revocação 0,82 e F1 0,80. A Tabela 6 detalha as métricas por classe.

A matriz de confusão (Tabela 7) esclarece o principal erro: as classes *sem defeito* e *cabeça queimada* confundem-se mutuamente — nesta execução, 11 das 15 etiquetas de *cabeça queimada* foram classificadas como *sem defeito*. As finas zonas claras entre as barras de um código íntegro

Tabela 6 – Métricas de classificação por classe (conjunto de teste sintético)

Classe	Precisão	Revocação	F1	N
Sem defeito	0,48	0,87	0,62	15
Cabeça queimada	0,75	0,20	0,32	15
<i>Ribbon</i> enrugado	0,82	0,93	0,87	15
Ponto queimado	1,00	1,00	1,00	15
Impressão clara	1,00	1,00	1,00	15
Pressão desigual	1,00	1,00	1,00	15
Cabeça suja	0,92	0,73	0,81	15
Macro / total	0,85	0,82	0,80	105

Fonte: elaboração própria (2026).

assemelham-se às linhas brancas verticais do defeito de elemento da cabeça, o que rebaixa a revocação de “cabeça queimada” (0,20). As demais classes, de assinatura visual mais distinta, são reconhecidas com F1 entre 0,81 e 1,00.

Tabela 7 – Matriz de confusão no teste (linha: classe verdadeira; coluna: classe prevista)

	sem defeito	cab. queimada	<i>ribbon</i> enrug.	ponto queim.	impr. clara	pressão desig.	cabeça suja
Sem defeito	13	1	1	0	0	0	0
Cabeça queimada	11	3	1	0	0	0	0
<i>Ribbon</i> enrugado	0	0	14	0	0	0	1
Ponto queimado	0	0	0	15	0	0	0
Impressão clara	0	0	0	0	15	0	0
Pressão desigual	0	0	0	0	0	15	0
Cabeça suja	3	0	1	0	0	0	11

Fonte: elaboração própria (2026).

Em testes ponta a ponta com o sistema em execução, o laudo é produzido por completo: o código é decodificado (por exemplo, EAN-13 e EAN-8 lidos corretamente pela biblioteca ZBar empacotada no projeto), o defeito é classificado e o diagnóstico traz a causa provável e a correção — para os defeitos de assinatura distinta, com confiança próxima de 1,00.

6.3 Multiagente *versus* abordagem monolítica

Esta é a comparação que transforma “usei ADK” em contribuição avaliável. Reportar as mesmas métricas para a arquitetura decomposta e para uma abordagem única (um só modelo/prompt fazendo tudo), discutindo confiabilidade, acerto por classe, interpretabilidade do laudo e latência. [A preencher.]

6.4 Análise e discussão

O sistema mostrou-se eficaz para os defeitos de assinatura visual distinta — ponto queimado, impressão clara e pressão desigual atingiram F1 igual a 1,00, e *ribbon* enrugado (0,87) e cabeça

suja (0,81) ficaram acima de 0,80. O ponto fraco concentra-se na confusão mútua entre “sem defeito” e “cabeça queimada”: as zonas claras entre as barras de um código íntegro imitam as linhas brancas verticais do elemento danificado, de modo que, dependendo do treino, uma das duas classes perde revocação. Isso evidencia a necessidade de mais dados dessas classes e de um pós-processamento que combine a saída da CNN com os indicadores de qualidade (por exemplo, exigir baixa uniformidade antes de declarar dano na cabeça). Frente aos trabalhos da Seção 3, que se detêm na detecção do defeito, o diferencial aqui é acoplar a classificação a um diagnóstico de causa fundamentado na documentação do fabricante.

6.5 Limitações

Ser explícito: os resultados acima são de um conjunto de teste **sintético e em distribuição** (mesma geração do treino); a avaliação de generalização para defeitos reais depende de um conjunto real — por exemplo, das bases do *Roboflow Universe* citadas na Seção 4.2 — e permanece em aberto. O treino de defeitos apoia-se em dados sintéticos, com risco de *domain gap*; a confusão entre “sem defeito” e “cabeça queimada” rebaixa a revocação de uma dessas classes (0,20 nesta execução); os números provêm de uma única execução de treino; a decodificação usa a biblioteca ZBar, que o projeto empacota localmente (dispensando instalação no sistema), embora códigos muito degradados ainda possam não ser lidos; o conjunto de captura real é pequeno (52 imagens, apenas 1D); o sistema não é verificador certificado (Seção 2.2); e há dependência das condições de captura e da generalização para outras impressoras/simbologias.

7 Conclusão

Este trabalho propôs um sistema de detecção de defeitos de impressão em etiquetas de códigos de barras que combina visão computacional e orquestração multiagente (ADK), servido por uma API única para aplicativo e web, e que vai além da nota tradicional ao sugerir a causa provável do defeito com base na documentação técnica do fabricante. **[Sintetizar aqui a resposta à questão de pesquisa da Seção 4, à luz dos resultados obtidos.]**

Principais contribuições entregues: [retomar da introdução as que se confirmaram].

Trabalhos futuros: ampliar o conjunto de dados com defeitos reais; estender a símbolos 2D (Data Matrix/QR) via ISO/IEC 15415; aproximar os indicadores estimados de um verificador certificado por calibração; otimizar a latência da orquestração; e realizar avaliação em ambiente de produção.

Referências Bibliográficas

ACKLEY, H. S. **System and method for detecting barcode printing errors**. 2017. U.S. Patent 9,826,106 B2. Concedida em 21 nov. 2017; prioridade 30 dez. 2014. Disponível em: <https://patents.google.com/patent/US9826106B2/en>. Acesso em: 13 jul. 2026. (Citado na página 10.)

AVRO, S. S.; RAHMAN, S. M. A.; TSENG, T.-L.; RAHMAN, M. F. A deep learning framework for automated anomaly detection and localization in fused filament fabrication. **Manufacturing**

Letters, v. 41, p. 1526–1534, 2024. DOI 10.1016/j.mfglet.2024.09.179. Anomalias em FFF por transferência de aprendizado (VGG/ResNet + YOLO). (Citado na página 10.)

GONZALEZ, R. C.; WOODS, R. E. **Digital Image Processing**. 4. ed. New York: Pearson, 2018. ISBN 978-0-13-335672-4. (Citado na página 9.)

Google. **Agent Development Kit (ADK): documentation**. 2025. Disponível em: <<https://google.github.io/adk-docs/>>. Acesso em: 13 jul. 2026. (Citado 3 vezes nas páginas 10, 14 e 15.)

Google. **Agent Development Kit: making it easy to build multi-agent applications**. 2025. Google Developers Blog. Anúncio de lançamento do ADK no Google Cloud NEXT 2025. Disponível em: <<https://developers.googleblog.com/en/agent-development-kit-easy-to-build-multi-agent-applications/>>. Acesso em: 13 jul. 2026. (Citado 3 vezes nas páginas 9, 10 e 13.)

ISO/IEC. **ISO/IEC 15426-1:2006: information technology: automatic identification and data capture techniques: bar code verifier conformance specification: part 1: linear symbols**. Geneva: International Organization for Standardization, 2006. Disponível em: <<https://www.iso.org/standard/43643.html>>. Acesso em: 13 jul. 2026. (Citado 2 vezes nas páginas 4 e 7.)

ISO/IEC. **ISO/IEC 15416:2016: automatic identification and data capture techniques: bar code print quality test specification: linear symbols**. Geneva: International Organization for Standardization, 2016. Disponível em: <<https://www.iso.org/standard/65577.html>>. Acesso em: 13 jul. 2026. (Citado 5 vezes nas páginas 4, 6 e 10.)

ISO/IEC. **ISO/IEC 29158:2020: information technology: automatic identification and data capture techniques: direct part mark (DPM) quality guideline**. Geneva: International Organization for Standardization, 2020. Disponível em: <<https://www.iso.org/standard/69411.html>>. Acesso em: 13 jul. 2026. (Citado na página 7.)

ISO/IEC. **ISO/IEC 15415:2024: automatic identification and data capture techniques: bar code symbol print quality test specification: two-dimensional symbols**. Geneva: International Organization for Standardization, 2024. VERIFICAR: edição vigente (2024) substitui a de 2011. Se citar a metodologia de 2011, ajuste ano e URL. Disponível em: <<https://www.iso.org/standard/76876.html>>. Acesso em: 13 jul. 2026. (Citado 4 vezes nas páginas 4, 7 e 10.)

JUN, X.; WANG, J.; ZHOU, J.; MENG, S.; PAN, R.; GAO, W. Fabric defect detection based on a deep convolutional neural network using a two-stage strategy. **Textile Research Journal**, v. 91, n. 1-2, p. 130–142, 2021. DOI 10.1177/0040517520935984. (Citado na página 10.)

SHUKLA, V.; SHUKLA, A.; K., S. P. S.; SHUKLA, S. A systematic survey: role of deep learning-based image anomaly detection in industrial inspection contexts. **Frontiers in Robotics and AI**, v. 12, p. 1554196, 2025. DOI 10.3389/frobt.2025.1554196. Disponível em: <<https://www.frontiersin.org/articles/10.3389/frobt.2025.1554196/full>>. Acesso em: 13 jul. 2026. (Citado 6 vezes nas páginas 9, 10 e 12.)

SZELISKI, R. **Computer Vision: Algorithms and Applications**. 2. ed. Cham: Springer, 2022. ISBN 978-3-030-34371-2. (Citado na página 9.)

TAO, X.; GONG, X.; ZHANG, X.; YAN, S.; ADAK, C. Deep learning for unsupervised anomaly localization in industrial images: a survey. **IEEE Transactions on Instrumentation and Measurement**, v. 71, p. 1–21, 2022. Art. no. 5018021. DOI 10.1109/TIM.2022.3196436. (Citado 6 vezes nas páginas 9, 10 e 12.)

Zebra Technologies. **ZM and ZT Series: resolving print quality issues**. 2024. Zebra Support – Base de Conhecimento. Disponível em: <<https://support.zebra.com/pt-BR/article/ZM-and-ZT-Series-Resolving-Print-Quality-Issues>>. Acesso em: 13 jul. 2026. (Citado 2 vezes nas páginas 8 e 9.)

Zebra Technologies. **ZT411/ZT421 industrial printer maintenance manual**. [S.l.], 2024. Disponível em: <<https://laserpros.com/img/manuals/zebra-manuals/zebra-zt411-zt421-industrial-printer-maintenance-manual.pdf>>. Acesso em: 13 jul. 2026. (Citado na página 7.)

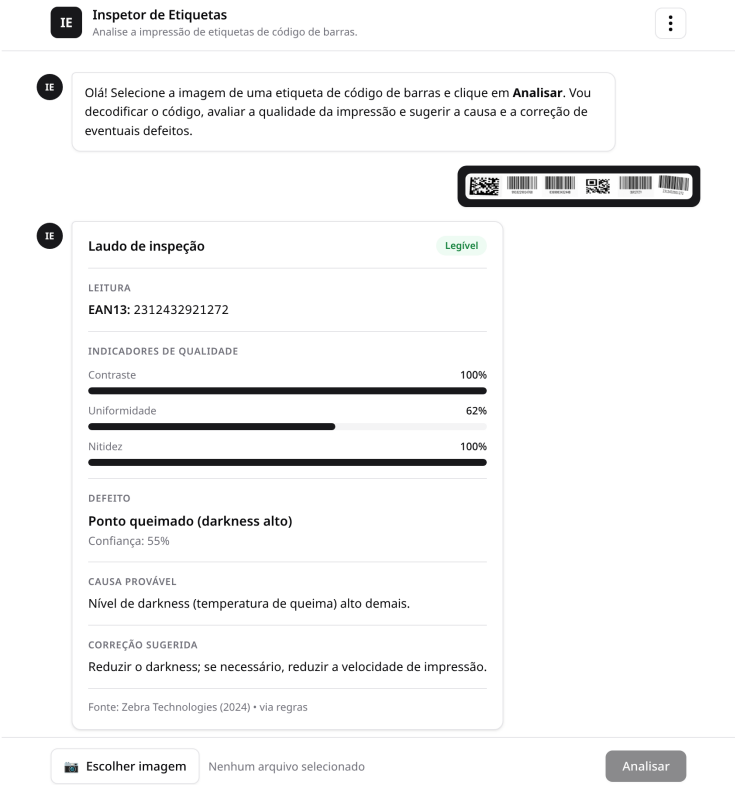
Zebra Technologies. **ZT411/ZT421 industrial printer user guide**. [S.l.], 2024. Documento P1106464. Inclui operação, manutenção e resolução de problemas de qualidade de impressão. Disponível em: <<https://www.zebra.com/content/dam/support-dam/en/documentation/unrestricted/guide/product/zt411-zt421-ug-en.pdf>>. Acesso em: 13 jul. 2026. (Citado 5 vezes nas páginas 4, 8 e 11.)

Figura 3 – Amostras do conjunto sintético de defeitos, uma classe por bloco



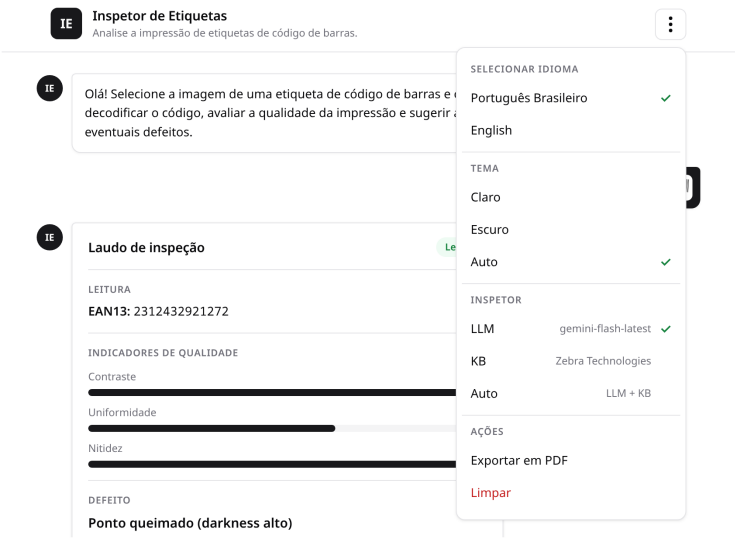
Fonte: elaboração própria (2026), gerada por `gerar_dataset.py`.

Figura 4 – Inspetor de Etiquetas em execução no chat: etiqueta enviada e laudo completo (leitura, indicadores, defeito, causa e correção)



Fonte: elaboração própria (2026).

Figura 5 – Menu de configurações do chat: seleção de idioma, tema, motor de diagnóstico (LLM/KB/Auto) e ações



Fonte: elaboração própria (2026).